怎么改

基于 8 轮实测与 20 次调用的逐段计时 · 2026-11-02 · 芝士

一、结论

慢不是单次调用慢，是调用次数多。每次调用中位 7 秒，一轮几十到上百次，于是十几分钟。
每次出第一个字中位 **1.73 秒**，20 次里 9 次不到 2 秒；请求发出前的准备几乎都在 0.4 秒以内。人看到的「卡住」不是哪一次特别慢，是很多次正常的等待叠起来。
所以靶子只有一个：**把次数降下来**。

二、依据

轮次总时长	调用次数	每次调用平均	首字出现	请求前准备
22.6s	6	3.8s	3.31s	1.79s
22.2s	7	3.2s	1.01s	0.07s
72.6s	10	7.3s	6.15s	1.87s
136.1s	20	6.8s	0.96s	0.07s
277.2s	29	9.6s	3.36s	0.35s
618.5s	79	7.8s	1.31s	0.07s
814.0s	119	6.8s	8.57s	0.12s
1011.3s	58	17.4s	2.52s	0.07s

8 轮真实会话，2026-11-01 下午。合计 2,975 秒、328 次调用。

三、能改的

1. 让一轮里的调用少而并行

根因：每次工具调用要一次完整的模型往返。三次独立的读文件、跑命令，串着发是三次往返，并行发是一次。
关键：并行的能力**本来就有**，不是平台限制——这一轮里我就同时发过两条独立命令，共用一次往返。所以这条不是补能力，是**让模型别不必要地串行**。
怎么做到：把「互不依赖的调用一次发出」写成硬规矩而不是建议；工具描述里也点明可以同时发。

2. 提醒不插在一轮中间

根因：提醒阈值是 600 秒沉默。核实过三次：它**插进正在跑的那一轮**，不是新开一轮。成本是一次额外往返加一句回话——不大，但白花。
改法：看板状态不靠叫醒会话也能更新。

四、不该动的

回复拦截：别放松。规则是「有人说话还没回时，除回话和读本房间消息外，工具一律拒绝」。这不是意外：原本只想提醒模型，但它会在工具边界读到消息后继续干原来的活，人什么都看不到。放松它，人就要干等一大片空白，还不知道在干什么。**这条不改。**
压上下文：零思考调用的等待与上下文体量无关（回归 r^2=0.01），压缩不省时间。
改缓存结构：实测每次已在读 38–46 万 token 的缓存，命中很充分，没有可捡的。
追长尾：超过 30 秒的调用只占 3.3%，成因还没查清，硬改是猜。

五、边界

第 1 条是按「一次往返 7 秒」推算的，**没做对照实验**。要坐实，得拿同一件事串行跑一遍、并行跑一遍，比总时长。
工具链那条（沙箱里没有编译器）单独写在另一份里。

口径：8 轮会话为 2026-11-01 下午；首字延迟 20 次调用实测，中位 1.73 秒、范围 1.31–8.83 秒。